

SIGIL: Compiling Agent Skills into Typed Harnesses

Jayanaka L. Dantanarayana
University of Michigan
USA
jayanaka@umich.edu

Lingjia Tang
University of Michigan
USA
lingjia@umich.edu

Savini Kashmira
University of Michigan
USA
savinik@umich.edu

Jason Mars
University of Michigan
USA
profmars@umich.edu

Abstract

AI-Integrated agents increasingly acquire capability from *skills*: prose procedure files loaded into a model’s context and run by a tool-calling loop. A skill is described to the runtime but never encoded in it, so the model re-derives its control flow on every run and may skip mandated verification. Across 30 skills and two model generations, a prose agent performs only 56% of the steps its own skill mandates, while producing artifacts that pass output checks. The remedy is known: write a harness, in which the procedure is program structure. However, hand-writing harnesses is tedious and discards the authoring surface that made skills succeed. To address this limitation, we introduce *Skill Compilation*, realized in *SIGIL*, which compiles a prose skill into an executable harness. At its center is *AG-IR*, a typed agentic intermediate representation separating model-owned cognition from code-owned mechanism. Compiled harnesses perform 86% of mandated steps, complete the full procedure 2.3× as often, and require 0.58× the tokens. Notably, the guarantee is model-independent: the harness holds at 86% across two model generations while prose swings from 56% to 68%.

Keywords

AI agents, compilers, intermediate representations, LLM programming

1 Introduction

Agents increasingly acquire specialized capabilities through *skills*: natural-language procedure specifications, such as Anthropic’s SKILL.md format [2], that are loaded into the agent context and interpreted by a tool-calling ReAct loop [10]. Skills have become a common unit for authoring, versioning, and distributing agent behavior.

However, a skill is available to the runtime only as prose. On every execution, the model must reconstruct the intended control flow, tool bindings, and stopping conditions. The runtime does not prevent the model from reordering steps, substituting tools, or omitting required checks. Across the 30 skills we study, we find that a prose agent executes only 56% of the steps mandated by its skill with gpt-4o and 68% with gpt-5. It completes the entire prescribed procedure in only 28% of runs.

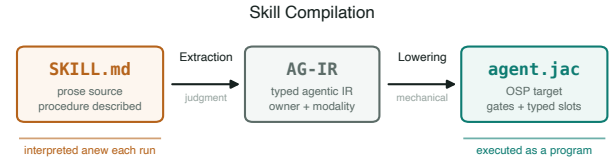


Figure 1: Skill Compilation. Extraction builds AG-IR, a typed graph in which every node has an owner and a modality, from a prose SKILL.md specification. Lowering then translates that graph mechanically into an executable agent harness.

A run that omits a mandated step is invalid because the prescribed procedure is part of the task specification. Artifact-level tests are therefore insufficient: they may accept an output that appears plausible without verifying how it was produced. The shortfall is not marginal. On 11 of the 30 skills we study, a prose agent executes fewer than half the mandated steps, and on two it executes none of them. For example, a financial-compliance report may contain reasonable figures and conclusions but remain invalid if the agent skipped required steps such as retrieving the authoritative records, independently recalculating the totals, or checking the result against the applicable policy. In such tasks, the procedure provides the evidence needed to establish the validity of the artifact.

This problem is already solved in engineering practice. A hand-written agent harness encodes the procedure as executable control flow: required tool invocations become bound calls, ordering constraints become program structure, and validation steps execute as ordinary statements. A harness does not omit the recalculation step, because that step is a line in a program rather than a sentence in a document. For the portion of the procedure represented in code, compliance no longer depends on the model reconstructing the instructions correctly. This enforcement, however, is obtained by replacing the skill with a separately implemented program.

This separation creates a gap between authoring and enforcement. Prose skills are accessible to domain experts but are not structurally enforced, whereas executable harnesses are enforceable but require software implementation. Converting each skill into a hand-written harness introduces three costs:

- *Engineering effort*: implementing a harness requires knowledge of both the domain procedure and the agent runtime, and this work must be repeated for each skill.



This work is licensed under a Creative Commons Attribution 4.0 International License.

- *Dual maintenance*: the prose skill and the executable harness become independent representations of the same procedure and may diverge as either is modified.
- *Loss of the authoring interface*: skills are portable, reviewable, and editable by domain experts, whereas hand-written harnesses move the procedure into a programming interface that is less accessible and more costly to maintain.

Prior work generally retains the prose execution model and adds runtime supervision, including formal monitors [1], action-boundary guards [3, 9], and rule languages evaluated during each run [8]. These approaches can detect or block disallowed actions, but detection does not ensure that a required action occurs. Moreover, when enforcement rules are maintained separately from the skill, the procedure and its enforcement mechanism can diverge. Existing systems therefore provide either an accessible authoring representation or structurally enforced execution, but not both from a single specification.

We formulate the translation from a prose skill to an executable harness as a compilation problem. A skill contains both *mechanism* and *judgment*. Mechanism includes fixed tool choices, control-flow constraints, validation checks, and operations that should execute identically across runs. Judgment includes open-ended decisions that require model inference. The mechanical portion can be represented as typed operations and explicit control flow, leaving only genuinely judgment-dependent operations to the model. Under this separation, the model supplies semantic judgment, while the program controls coordination and execution.

We propose **Skill Compilation**, which translates a prose procedure specification into an executable agent harness. A mandated step then executes because it is represented in the program structure rather than because the model correctly interprets a prose instruction. We realize this approach in **SIGIL** (Skill Intent Grounding and Intermediate Lowering), a skill compiler with two responsibilities: grounding the extracted procedure in the source skill and lowering the resulting representation without introducing additional model judgment. SIGIL uses one intermediate representation and two compilation stages, as shown in Figure 1.

AG-IR is a typed intermediate representation for agent procedures. Its nodes record two things a prose skill leaves implicit: an *owner*, stating whether code or the model executes the step, and a *modality*, stating how binding the original instruction was. Each node also retains provenance to the sentence that mandates it, so any part of the compiled program can be traced back to the skill.

The compiler itself has two stages. **Extraction** builds an AG-IR specification from the skill under a faithfulness constraint: each admitted rule must be supported by a verbatim quotation from the source, and compile gates reject specifications that fail the required consistency and validity checks. **Lowering** is deterministic. Each AG-IR primitive has a fixed translation into an Object-Spatial Programming (OSP) [6] agent. Consequently, Lowering introduces no additional model interpretation, and structurally encoded steps cannot be omitted, reordered, or redirected to a different tool by the executing model.

We implement SIGIL in Jac [5], a production-grade Python superset with native OSP constructs. Model-owned operations use byLLM’s meaning-typed abstractions [4], which express each model

invocation as a lightweight typed declaration of intent. We evaluate SIGIL on 30 agent skills, running each nine times per arm on each of two models. Compiled harnesses execute 86% of mandated steps, compared with 56% for the standard prose baseline, and complete the full prescribed procedure 2.3× as often. At the median, compiled execution consumes 0.58× the tokens of prose execution. These results show that a procedure can be authored once in prose while its mechanically specified requirements are enforced by the generated program.

This paper makes the following contributions:

- **Skill Compilation**, a compilation paradigm that translates a prose procedure specification into an executable agent harness, making mandated steps part of the program structure.
- **AG-IR**, a typed agentic intermediate representation that separates model-owned cognition from code-owned mechanism through typed primitives, node ownership, modality, and provenance to the source skill.
- **SIGIL**, a compiler whose Extraction stage grounds every admitted rule in a verbatim quotation and applies compile gates that reject unfaithful specifications, and whose deterministic Lowering stage translates accepted AG-IR programs without additional model judgment.
- **An evaluation** across 30 agent skills showing that compiled harnesses execute 86% of mandated steps, compared with 56% for prose, and complete the full procedure in 65% of runs, compared with 28%, while using 0.58× the median token cost. Compiled compliance remains at 86% across two model generations, while prose compliance changes from 56% to 68%.

2 Motivation

To make the gap concrete, we examine three skills from our suite, each drawn from a different family, and compare what the skill mandates against what the prose agent actually did across its runs. In each case we then show the harness form of the same step. The three failures are not exotic: they are a check that was claimed rather than run, an API call that was described rather than issued, and a staged procedure that was collapsed into a single act of writing.

2.1 Three ways prose fails

The check that never ran. The verification-before-completion skill governs how an agent may claim that work is done. Figure 2 reproduces its core. The skill is emphatic and unambiguous: it names an “Iron Law” (lines 3–5), states the consequence of violating it (line 7), and specifies a five-step gate function (lines 11–18) that must precede any claim of success, closing with an explicit warning that skipping a step is dishonesty rather than efficiency (line 20). It is difficult to write a procedure more clearly than this.

However, it did not help. In our runs the prose agent routinely wrote “all tests pass, build succeeds” directly into the deliverable without a fresh run. A claim in an output document is not the same as having run the check, and across the scored runs the linter, build, and zero-failure confirmations were absent on nearly every one. The rule the skill states most forcefully, that a claim must be backed by a fresh execution, is the rule the model was most likely to drop.

```

1  ## The Iron Law
2
3  NO COMPLETION CLAIMS WITHOUT FRESH VERIFICATION EVIDENCE
4
5  If you haven't run the verification command in this
6  message, you cannot claim it passes.
7
8  ## The Gate Function
9
10 BEFORE claiming any status or expressing satisfaction:
11 1. IDENTIFY: What command proves this claim?
12 2. RUN: Execute the FULL command (fresh, complete)
13 3. READ: Full output, check exit code, count failures
14 4. VERIFY: Does output confirm the claim?
15   - If NO: State actual status with evidence
16   - If YES: State claim WITH evidence
17 5. ONLY THEN: Make the claim
18
19 Skip any step = lying, not verifying

```

Figure 2: The core of the verification-before-completion skill, abridged. The procedure is stated precisely, and every word of it is advisory at run time.

Prose satisfies 30% of this skill’s mandates; the harness satisfies 84%.

Notably, the reason is visible in Figure 2 itself. Steps 2 and 3 of the gate function describe running a command and reading its exit code, which are actions whose outcome is determined by their inputs. Nothing in the prose form causes them to happen. The document is consulted by the same process that is supposed to obey it, and that process can satisfy its own reading of the instruction by writing a sentence that asserts the outcome.

The call that was narrated, not made. The gh-issues skill mandates fetching state from the GitHub REST API before acting on it: the issue list, the inline review comments, and the pull-request conversation, each from a named endpoint. The prose agent described the requests it would make, then reasoned from what it could already see in its context, and missed the fetch mandates on every scored run. The resulting summaries were fluent and plausible, which is precisely the problem: they were derived from stale prompt content rather than from the live state the skill required the agent to retrieve. Prose satisfies 20% of this skill’s mandates; the harness satisfies 100%.

The procedure collapsed into one deliverable. The brainstorming skill mandates a staged interaction: propose two or three approaches with trade-offs, present the design section by section and seek approval after each, then write the design document to a specified path and commit it. The prose agent folded the entire procedure into a single authored document, skipping the alternatives, never pausing for approval, and never committing the file. The output was a reasonable design document, so nothing about it signals that the collaborative procedure the skill exists to enforce did not happen. Prose satisfies 40% of this skill’s mandates; the harness satisfies 99%.

2.2 What the failures have in common

Three observations follow, and together they motivate the design.

The failures are procedural, not linguistic. In each case the model demonstrably understood the instruction. It can restate the mandate accurately when asked, and it often narrates an intention to perform the step. What it does not do is perform it. The deficiency is in

```

1 obj Evidence {
2   has command: str = "";
3   has ran: bool = False;
4   has exit_code: int = -1;
5   has failure_count: int = -1;
6   has evidence_ok: bool = False;
7 }
8
9 can n_gather_evidence with GatherEvidence entry { # step 2: RUN
10   cmd = self.plan.verify_command.strip();
11   res = _run_cmd(cmd, ""); # a real subprocess
12   self.evidence.ran = bool(res[0]);
13   self.evidence.exit_code = int(res[1]);
14   visit [-->][?ReadEvidence];
15 }
16
17 can n_read_evidence with ReadEvidence entry { # step 3: READ
18   fc = int(_count_failures(self.evidence.output,
19                           self.evidence.exit_code));
20   self.evidence.failure_count = fc;
21   self.evidence.evidence_ok = bool(self.evidence.ran
22   and self.evidence.exit_code == 0 and fc <= 0);
23   visit [-->][?AssessEvidence];
24 }

```

Figure 3: Steps RUN and READ of the same gate function, compiled. The abilities fire on node entry, and the evidence they produce is populated by code rather than asserted by the model.

execution, not comprehension, so remedies that clarify the prose address the wrong layer.

The failures are defects, and artifact tests cannot see them. In all three vignettes the deliverable is plausible and would pass an output check: a report that claims tests pass, a summary of review comments, a design document. A run that skipped the mandated step is nonetheless wrong, because in these domains the procedure is the product. Evaluating such a run by inspecting its output does not measure a milder notion of success; it applies the wrong oracle and certifies defective work.

The failures concentrate on mechanical steps. The dropped mandates are the fetch, the check, and the commit. These are exactly the steps whose execution is a function of their inputs, which is to say exactly the steps a program encodes without difficulty. The steps that survived prose execution are the open-ended ones, where the model was asked to write something and did.

2.3 What a harness does instead

Figure 3 shows steps 2 and 3 of the same gate function as executable structure. The comparison with Figure 2 is the argument of this paper in miniature.

Importantly, three properties distinguish the compiled form. First, the abilities are bound to node entry (with `GatherEvidence` entry), so they execute when the walker reaches the node. The model is never asked whether to run the command, because nothing in the graph consults it on that question. Second, the `Evidence` object is populated by code: `ran` is set from the result of an actual subprocess, and `exit_code` and `failure_count` are read from its output. The model cannot set these fields, so it cannot assert that a check passed. Third, `evidence_ok` is computed rather than claimed, and downstream nodes consume it, so a success claim is structurally unreachable when nothing ran.

The remaining two vignettes are remedied the same way. For gh-issues, the fetch steps become code-owned nodes whose bodies issue a real request against each endpoint, so the data is retrieved before any downstream reasoning and the call cannot be narrated away. For brainstorming, each stage becomes its own node that

fires when the walker reaches it, so the sequence executes instead of being summarized.

In essence, none of this requires a research contribution. It requires a program. That is the point of Section 1’s second observation: enforcement is a solved problem, and what is missing is a way to obtain the program from the artifact the domain expert actually wrote. Writing these three harnesses by hand was tedious, demanded an engineer familiar with both the domain and the runtime, and left a second artifact that will drift from the skill the moment either is edited. Across hundreds of skills, hand-writing is not a strategy.

This is what motivates treating the problem as compilation. The remaining question is what vocabulary can express both halves of a skill, and what discipline keeps the translation faithful, since a front end that quietly rewrites the procedure would convert a guaranteed step into a guaranteed deviation. Specifically, we ask:

- (1) What is the smallest primitive vocabulary in which every step of a prose skill has a natural target, spanning both open-ended cognition and deterministic mechanism?
- (2) How can the assignment of steps to primitives be made faithful to the source, given that the front end must use a model to read prose?
- (3) What must hold of the lowering so that a structurally enforced step cannot be skipped, reordered, or re-tooled at run time?
- (4) How much of a real skill can be enforced this way, and what does the residue cost?

3 AG-IR

3.1 Design requirements

The representation sits between a document written for humans and a program executed by a machine, which imposes four requirements. An ideal agentic IR should: (1) **span cognition and mechanism**, so that both an open-ended authoring step and a fixed tool invocation have a natural target and neither must be encoded as the other; (2) **make the executor explicit**, so that it is a checkable property of the IR, rather than an emergent property of generated code, whether a given step depends on a model; (3) **preserve the source’s binding force**, since a skill’s “never” and its “may” must not lower to the same construct; and (4) **retain provenance**, so that every element of the compiled program can be traced to the sentence that required it, and any step of a run can be traced back the same way.

3.2 The node alphabet

AG-IR is a typed graph over four primitive families, listed in Table 1. The *Mind* family is model-owned and ordered from tightest to loosest output space, which makes the choice among its members an explicit engineering decision rather than an accident of phrasing. Classifying which operation a request calls for is a GEN-ENUM over a fixed set, not free generation, and typing it that way removes an entire class of deviation. The *Flow* family carries control. The *Boundary* family separates pulling world state in from pushing effects out, which matters because the two have different idempotence and different auditing needs. The *Code* family is deterministic and free.

Table 1: The AG-IR node alphabet and its lowering targets.

Node	Owner	Lowens to
<i>Mind</i> , tightest to loosest		
GEN-ENUM	model	typed slot over a fixed set
GEN-FILL	model	typed slot returning an object
GEN-EDIT	model	typed slot mutating content
GEN-RAW	model	free-form typed slot
<i>Flow</i>		
ROUTE	either	child nodes plus a dispatch
LOOP	code	bounded walker iteration
SPAWN	code	parallel sub-walkers
CALL	code	invocation of a sub-graph
<i>Boundary and Code</i>		
SENSE	either	read, grep, or fetch (idempotent)
ACT	either	write a file, emit a deliverable
CODE	code	a pure function of its inputs
TERMINAL	code	the single exit, serializes results

3.3 The Owner test

In essence, assigning a step to a family is governed by one rule, which we call the **Owner test**: *is this step’s output a function of its inputs?* If it is, code owns the step, and it lowers to structure that executes unconditionally. If it is not, because the step requires taste, open-ended synthesis, or a judgment the inputs do not determine, the model owns it and it lowers to a typed slot. Running a recalculation script is a function of the workbook, so code owns it. Deciding whether a design reads as compelling is not, so the model owns it.

Importantly, the test is deliberately narrow. It does not ask whether a model *could* perform the step, because a model can perform almost any step; it asks whether the step’s result is determined. This matters for the guarantee, because model-owned nodes are the only places a run can deviate, so the proportion of the procedure that survives the Owner test as code-owned is the proportion that becomes deviation-proof.

3.4 Modality becomes structure

Each rule carries the binding force of the prose that stated it, and each modality lowers to a different structural form.

- **Mandatory** becomes an ability that fires on node entry. The walker cannot pass through the node without executing it, so the model is never consulted about whether to perform the step.
- **A mandated procedure** becomes a code-owned gate that performs the work, runs the script, drives the library, and records its outcome in the node trace.
- **Forbidden** becomes an *absent path*. Rather than instructing the model not to take an action, the compiler emits no node that performs it. A model cannot route to a node that does not exist, which is a stronger property than any instruction.

- **Discretionary** becomes a typed verdict that the surrounding code consumes, so the model’s latitude is preserved but its expression is constrained to a value the program can act on.

Consequently, only genuine cognition survives as a model-owned slot. A prose agent executes the skill’s control-flow graph implicitly, re-deriving it on every run; a compiled harness executes the same graph with the coordination compiled to code, and consults the model only where the Owner test says it must.

3.5 Cohesion, and the limit of compilation

Not every mandate should be compiled as tightly as possible, and the IR records this explicitly through a *cohesion* attribute on each rule. A rule is **ATOMIC** when it names a single fixed move that is safe to lower as its own node. A rule is an **INVOKE-UNIT** when it describes an adaptive, tool-using activity, such as exploring a repository until the relevant module is found or iterating until a test passes. An **INVOKE-UNIT** must remain one cognitive unit equipped with a tool belt, because decomposing it into a fixed sequence of calls destroys the observe-and-adapt loop that makes it work.

In essence, this is the honest boundary of the approach. Compilation converts a mandate into structure exactly when the mandate’s execution is determined; where a skill genuinely requires the model to look at what came back and decide what to do next, the compiler’s job is to preserve that latitude rather than to eliminate it. Over-constraining judgment degrades the result just as surely as under-constraining mechanism forfeits the guarantee. Section 5 shows the cost of honoring this boundary: on skills built around an **INVOKE-UNIT**, the harness faithfully runs a tool-using loop that the prose agent frequently skipped, and consequently spends more, not less.

3.6 Four views and the provenance spine

Finally, the same graph is read in four ways: control flow, dataflow, knowledge residency, and human gates. Extraction emits each view separately and the assembler joins them on a spine of rule identifiers. The result is that every node records the rules it realizes, and every rule records the span of the skill it came from. Provenance therefore runs unbroken from a sentence in the **SKILL.md**, through the rule extracted from it, to the node that realizes it, to the compiled ability, and finally to the line in a run’s node trace showing it executed. This chain is what makes a compiled run auditable in a way that a prose run is not, and Section 5 relies on it directly.

4 The SIGIL Compiler

SIGIL compiles a prose skill into an executable harness in two stages over the AG-IR of Section 3. *Extraction* builds an AG-IR specification from the skill, and *Lowering* translates that specification into a running program. The two stages differ in one respect that governs the whole design: reading prose requires a model, so Extraction contains one, whereas Lowering contains none. Confining model judgment to a single stage, where it is checked, is what makes the emitted artifact auditable, because a reviewer who wants to know what a harness does can read the IR rather than the code generator.

4.1 Extraction: from a skill to AG-IR

Extraction is the compiler’s most delicate region. A stage that quietly rewrote the procedure would convert a guaranteed step into a guaranteed deviation, and would do so invisibly, since the output would still be a clean-compiling harness. It is therefore built so that model judgment is always subordinate to a lightweight, deterministic check. The organizing principle is that *the model proposes and code disposes*.

The specification loop. The first phase extracts the skill’s rules and is the anchor for everything downstream. A model-owned slot proposes candidate rules, each carrying a modality read from the prose’s deontic vocabulary, a kind, a cohesion, and a verbatim quote of the span it came from. Code then disposes of the proposal in three ways. *Grounding* requires every candidate rule to quote the skill verbatim; a rule whose quote does not appear in the source is dropped mechanically, without consulting the model, so hallucinated obligations cannot survive and the loop can never declare victory over an ungrounded specification. *Coverage* runs three critics that search independently for obligations the extraction dropped, since the failure mode of a single extractor is silent omission rather than visible error, and each critic’s proposals pass the same grounding filter. A *deontic audit* compares each rule’s assigned modality against the language of its quote, catching drift in both directions, because hardening a “may” into a “must” is as unfaithful as softening a “never” into a preference. Consequently, the loop iterates until the specification is sound, complete, and free of modality drift, with dry-run detection and a hard iteration cap so that a pathological skill fails rather than spins. It then *freezes* the rule set, and every later phase is audited against the frozen set, so nothing downstream can quietly introduce or discard an obligation.

The workflow spine. Next, each frozen rule is typed into the AG-IR alphabet, the Owner test is applied, and the control-flow graph is wired. A code-side validator then enforces three properties the model is prone to violate. Every mandatory rule must be realized by some node, so an obligation cannot be typed away. An **INVOKE-UNIT** must not be shredded into a fixed sequence of calls, which protects adaptive tool use from over-compilation. Prohibitions must become constraints rather than paths, since emitting a node for a forbidden action and instructing the model to avoid it reintroduces exactly the failure mode compilation exists to remove.

Annotator flows and assembly. Three further views are produced in parallel over the frozen specification. The dataflow view determines which values move between nodes and therefore what the walker carries. The knowledge view scopes reference material, deciding which node sees which portion of the skill’s supporting text, which is what allows each model-owned slot to receive a small, relevant context instead of the whole document. This view also performs a sort that matters for faithfulness: a code snippet in a skill is either a runnable tool body or grounding material to be shown to the model, and treating one as the other silently changes what the harness does. The human-gate view marks the points at which the procedure requires a person to approve or decide. Finally, an assembler joins the views on the rule identifier spine and persists provenance onto every node, so the chain from skill sentence to compiled ability is complete before any code is generated.

Table 2: The compile gates and what each rejects.

Gate	Rejects
G1 standalone	a tool or knowledge entry that <i>points at</i> content instead of embodying it, such as an instruction to consult another file
G3 artifact boundary	a mandated deliverable that no node actually writes
G4 compile oracle	an AG-IR that does not lower to a clean-compiling module, checked by running the real Lowering stage and type checker
G5 STRUCT-COV	a mandatory rule folded into another slot’s interior, leaving it model-dependent
G6 human gates	a route that consumes human feedback but leaves the decision to model judgment

Compile gates. However, an assembled AG-IR is not yet admitted. Six gates decide whether it may be lowered, and each rejects a specific way in which a plausible-looking IR fails to be a faithful compilation of its source. Table 2 lists them.

Notably, two of these deserve elaboration. **G1** exists because a skill frequently delegates to its own supporting files, and an extraction that preserves the delegation produces a harness that is not self-contained. The gate enforces a simple test: if the `SKILL.md` and its directory were deleted, would the compiled harness still perform the procedure? Content must be embodied in the IR, not referenced from it. **G4** is the compile oracle, and it is what makes Extraction’s output falsifiable. A candidate IR is passed through the real Lowering stage and the language’s type checker on every compilation, not as a periodic test, and an IR that produces an empty or non-compiling module is rejected outright. Because the oracle runs the same translation used in production, Extraction cannot emit an IR that is well-formed on paper but meaningless in practice.

STRUCT-COV. **G5** deserves separate treatment because it is also the compiler’s principal diagnostic. For every mandatory rule of kind step or deliverable, STRUCT-COV determines by static analysis of the *emitted module* how the harness realizes it: **gated**, meaning a code-owned ability realizes the rule so it holds regardless of the model; **slotted**, meaning a dedicated model-owned slot realizes it, so the step is structurally reached and individually observable but its content depends on the model; **monolithic**, meaning the rule is folded into another slot’s interior with no dedicated structure of its own; or **missing**, meaning nothing in the graph realizes it. Monolithic is the dangerous outcome, because the compilation appears to succeed while the rule retains exactly the risk profile it had as prose. Monolithic and missing rules fail the gate and are reported by identifier, naming the node to strengthen. STRUCT-COV is best understood as a coverage report rather than a prediction: it states how much of a particular skill a particular compilation actually enforced, in the way that code coverage states which lines a test suite exercised.

Repair. Finally, a gate failure routes to a bounded repair pass in which each view fixes its own errors, after which the gates run again. Repair is scoped, so a knowledge-scoping error cannot trigger a rewrite of the control-flow graph. When repair cannot resolve

the failures, Extraction returns its issues rather than a weakened artifact. Nothing unfaithful is persisted, and compilation fails loudly with rule-level diagnostics.

4.2 Lowering: from AG-IR to an agent harness

Lowering transpiles an admitted AG-IR into an executable OSP module. It contains no model call and makes no choices. Its governing invariant is stated negatively, because that is how it is enforced: *Lowering never needs to decide anything*. If translating a construct would require a judgment call, the IR was underspecified and the fault lies in Extraction, which is where the fix belongs.

Translation rules. In contrast to Extraction, this stage has no latitude. Object-Spatial Programming provides the target abstractions directly, so the mapping is close to one-to-one. An AG-IR node becomes a node archetype in the emitted graph. An AG-IR edge becomes a graph connection, which fixes the walker’s traversal order and therefore the procedure’s sequence. Data that moves between steps becomes walker state. A mandatory rule becomes an ability bound to node entry, so it executes when the walker arrives rather than when the model elects to invoke it. Model-owned nodes become typed by `llm()` slots, using meaning-typed abstractions [4] so that each slot has a declared return type and receives only the context its knowledge scope assigns. The tightness ordering within the Mind family is preserved by that return type: a GEN-ENUM node lowers to a slot whose type is a fixed enumeration, so an out-of-range answer is a type error rather than a silent deviation, while a GEN-RAW node lowers to a slot returning free text. Code-owned nodes lower to ordinary functions, subprocess invocations, or library calls, with no model involvement at any point.

Notably, two constructs are where naive translation tends to fail. A ROUTE lowers to a set of child nodes plus a dispatch over them, so the branch structure is carried by the graph rather than by an instruction and the set of reachable continuations is fixed at compile time. A forbidden rule lowers to nothing at all, which is the intended behavior: the guarantee comes from the absence of a path, not from the presence of a prohibition.

The emitted runtime. Every compiled module embeds a small runtime whose purpose is to make the run observable and its failures honest. Code-owned work does not appear in the model’s token stream, so without instrumentation an auditor inspecting a transcript would see less evidence of work from the harness than from a prose agent that merely narrated its intentions. The runtime therefore emits a *node-path trace*: one record per node entry, naming the node and the outcome of any code-owned action it performed. This trace is the harness’s action log and the counterpart of the tool calls visible in a prose agent’s transcript, and Section 5 uses it as evidence. Importantly, three further behaviors make runs honest rather than merely observable. A walker that terminates without reaching its terminal node reports an explicit incomplete outcome instead of returning whatever it has, so a truncated run cannot be mistaken for a successful one. A step budget aborts runaway loops and names the node that exceeded it. A per-call token log records the cost of every model call, which is the basis of the cost measurements in Section 5.

Ejection. A compiled harness can be emitted as a single self-contained file with an interpreter directive and a command-line shim, so that running the skill requires neither the compiler, nor a graph store, nor a session. As a result, the unit of distribution remains a single file, as it was when the skill was prose, except that the file is now a program. The execution model is also unchanged from the user’s perspective: the artifact takes a task in natural language and runs it, and the model used for the model-owned slots is selected at run time, so the same compiled harness runs on a frontier model or a small local one without recompilation. Section 5 exploits this to hold the harness fixed while varying the model.

4.3 Implementation

We implement SIGIL in Jac [5], a Python superset with native OSP constructs, in approximately 4,400 lines across the two stages. Lowering is a single transpiler of roughly 1,650 lines that maps each IR construct to its target form and injects the runtime helpers verbatim. Extraction comprises the specification loop, the workflow spine, the annotator flows, the assembler, the gates, and the repair pass. The compile oracle invokes the real Lowering stage and the Jac type checker, so the two stages are exercised against each other on every compilation.

5 Evaluation

We evaluate compiled harnesses against the standard way a skill is consumed today. Specifically, we aim to answer the following research questions:

- **RQ1** How faithfully does a compiled harness execute a skill’s mandated procedure, relative to a prose agent given the same skill?
- **RQ2** How does that advantage vary with the capability of the underlying model?
- **RQ3** What does compilation cost at run time?
- **RQ4** How much of a skill does the compiler enforce, and where does it fail?

5.1 Experimental setup

Skills. We use 30 agent skills spanning three families, summarized in Table 3. *Document and tooling* skills produce an artifact through a prescribed library or tool, and include both document production (*docx*, *xlsx*, *pptx*) and developer tooling (*mcp-builder*, *gh-issues*, *webapp-testing*). *Software process* skills (*brainstorming*, *executing-plans*, *verification-before-completion*) govern how work is carried out rather than what is produced, and carry the most mandates per task. *Governance and compliance* skills (*soc2-system-description*, *iso27001-internal-auditor*, *hipaa-compliance*) require evidence to be gathered and cited. Twenty-six of the 30 are drawn from public skill collections; the four governance and compliance skills were authored for this study. Notably, the families differ in how mechanical they are, which is what lets us observe where compilation pays and where it does not: the gate share in Table 3 is the fraction of mandatory steps the compiler realized as code-owned gates, and it ranges from 32% to 44% across families.

Conditions. The baseline, which we call *prose*, is the standard consumption path: the `SKILL.md` injected into a ReAct loop equipped

Table 3: The skill suite. Mandates are the MUST steps in a skill’s reference procedure for one task; gate share is the fraction the compiler realized as code-owned gates.

Family	Skills	Mandates/task	Gate share
Document & tooling	13	10.7	32%
Software process	13	16.5	44%
Governance & compliance	4	7.2	41%
All	30	12.8	37%

with file and shell tools. The treatment, which we call *harness*, is the compiled artifact. Both arms delegate every authoring step to the same model through the same interface, so the comparison isolates the effect of compiling the coordination and does not confound it with a difference in writing ability.

Tasks and repetition. Each skill has three process-scenario tasks, chosen to exercise the skill’s procedure rather than a single happy path, and each task is repeated three times, giving nine runs per skill per arm. Every run executes in an isolated sandbox with its own working directory and its own throwaway repository, so that runs cannot observe or disturb one another.

Models. We run the entire suite through the OpenAI APIs for gpt-4o and gpt-5 [7] at default sampling settings, so each skill is run nine times per arm on each model. Runs execute on a single workstation, since both arms are API-bound and neither performs local inference. Because a compiled harness selects its model at run time, the harness artifact is byte-identical across the two sweeps, which is what makes RQ2 a clean comparison.

5.2 One metric, and why

We report a single measure of whether a run did the work, and we do not report artifact correctness as a separate axis. This is a deliberate departure from common practice and follows from the position argued in Section 2: a run that fails to perform a mandated step is wrong, and an artifact test that passes such a run has not measured a second dimension of quality, it has failed to detect a defect. Procedure execution is therefore the correctness criterion rather than a proxy for it.

To measure it, each skill’s prose is distilled into a reference procedure of mandatory steps. A run is scored by **Applicable-Mandate Compliance (AMC)**: of the mandated steps that *applied* to that run, the fraction the agent actually *performed*. A judge reads the complete run and assigns each mandate exactly one of four statuses: *followed*, meaning the step was performed or a code-owned gate for it fired; *violated*, meaning it was attempted incorrectly; *missed*, meaning it applied and was not performed; and *not applicable*, meaning the run never reached the point where the mandate applies. AMC is the followed count over the sum of the first three.

The denominator is *applicable* rather than *all*, which matters for fairness. When a run correctly branches away or stops early, the downstream steps it never reached are excluded, so a correct decision to skip a branch is never charged as a miss. The judge reads the token stream and, for a compiled harness, the node-path trace described in Section 4.2. Including the trace is necessary rather than

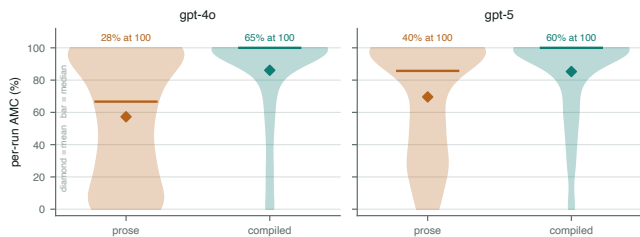


Figure 4: Per-run AMC pooled across all 30 skills. The prose arm spreads across the entire range with substantial mass at low compliance; the harness concentrates at full execution. Diamonds mark means and bars mark medians.

generous: code-owned work never appears in the token stream, so the trace is the harness’s action log and is the direct counterpart of the tool calls visible in a prose transcript. Section 5.7 reports what happens when this credit is removed entirely.

One objection deserves a direct answer: does the harness win by performing the steps while producing worse artifacts? The design answers this structurally rather than with a second metric. Both arms route every authoring step to the same model through the same typed interface, so compilation changes which steps run and in what order, never how the prose is written. What compilation removes is the option to skip.

5.3 RQ1: procedure execution

On gpt-4o, a prose agent performs 56% of the steps its own skill mandates, while the compiled harness performs 86%. The harness matches or exceeds prose on all 30 skills, with 28 strict wins, 2 ties, and no losses.

Notably, the distribution is more informative than the mean. As Figure 4 shows, the prose arm does not fail uniformly; it spreads across the whole range, which is the signature of a procedure that is sometimes followed and sometimes not, with no way for an observer to tell which happened. The harness concentrates near full execution. Counting only runs that performed the entire applicable procedure, prose succeeds on 28% of runs and the harness on 65%, a factor of 2.3.

Figure 5 breaks this down by skill. The largest gaps fall on mechanism-heavy skills, where most mandates are fixed tool calls and ordered checks, and the smallest on judgment-heavy skills, where most mandates ask for open-ended authoring that a prose agent performs simply by writing. This is the behavior the Owner test predicts: compilation converts exactly the determined steps into structure, so the advantage tracks how much of a skill is determined.

Key Takeaways.

- (1) **Prose loses the procedure, not the prose.** A prose agent performs 56% of mandated steps while producing plausible artifacts throughout, so the deficiency is invisible to any check that inspects only the output.
- (2) **Compilation makes full execution the common case.** Complete procedures rise from 28% to 65% of runs, and no skill is worse off.

5.4 RQ2: the capability axis

Moving from gpt-4o to gpt-5, the prose agent improves substantially, from 56% to 68%. The compiled harness does not move: 86% to 86%. Figure 6 shows the two trajectories.

This is the result we consider most consequential, and the explanation is structural. In the prose arm, the model is responsible for reconstructing the procedure on every run, so procedure execution is a capability that improves as models improve. In the harness, the graph is responsible, and the graph is the same artifact in both sweeps. The model still authors the content of every model-owned slot, but it no longer decides which steps happen, so improvements in model capability have nothing to contribute on this axis.

In essence, this inverts the usual argument for waiting. The advantage of compiling is 30 points on the weaker model and 17 on the stronger one, so compilation matters most exactly where models are weakest and cheapest to run. A team choosing a small model for cost or latency reasons is the team that benefits most.

On gpt-5 the harness wins on 21 skills, ties on 5, and loses on 4. The losses are concentrated in judgment-heavy skills where a strong model executes the prose procedure well and the compiled graph’s residual model-owned slots have no structural advantage to contribute. We regard this as the predicted behavior rather than an anomaly: where a skill is mostly judgment, there is little to compile, and a sufficiently strong model needs little help.

Key Takeaways.

- (1) **The guarantee is model-independent.** The harness holds at 86% across both models because structure, not capability, carries the procedure.
- (2) **Compilation substitutes for capability.** The advantage narrows from 30 points to 17 as the model strengthens, so the technique pays most on the cheap models teams actually want to deploy.

5.5 RQ3: cost

Compilation reduces cost at the median. Figure 7 gives the per-skill ratio of harness tokens to prose tokens. Across the 32 skills for which both arms produced token accounting, the harness costs 0.58× what the prose agent costs per run, and it is cheaper on 24 of them. The savings come from the same property that produces the compliance result: a compiled harness does not re-read a long prose document on every step, and each model-owned slot receives only the context its knowledge scope assigns, so the tokens spent re-establishing the procedure disappear. On mechanism-heavy skills the reduction is dramatic, reaching 0.02× on using-git-worktrees, where nearly the whole procedure is code.

However, the minority of skills where the harness costs more is more interesting, and we report it plainly. The extreme is using-superpowers at 7.64×, followed by systematic-debugging at 3.09×. These are skills organized around an adaptive tool-using activity, the INVOKE-UNIT of Section 3. The compiler preserves such an activity as a genuine tool-using loop rather than decomposing it, and a genuine loop consumes tokens. The prose agent frequently truncated or skipped the same loop, which is cheaper and is also the failure the

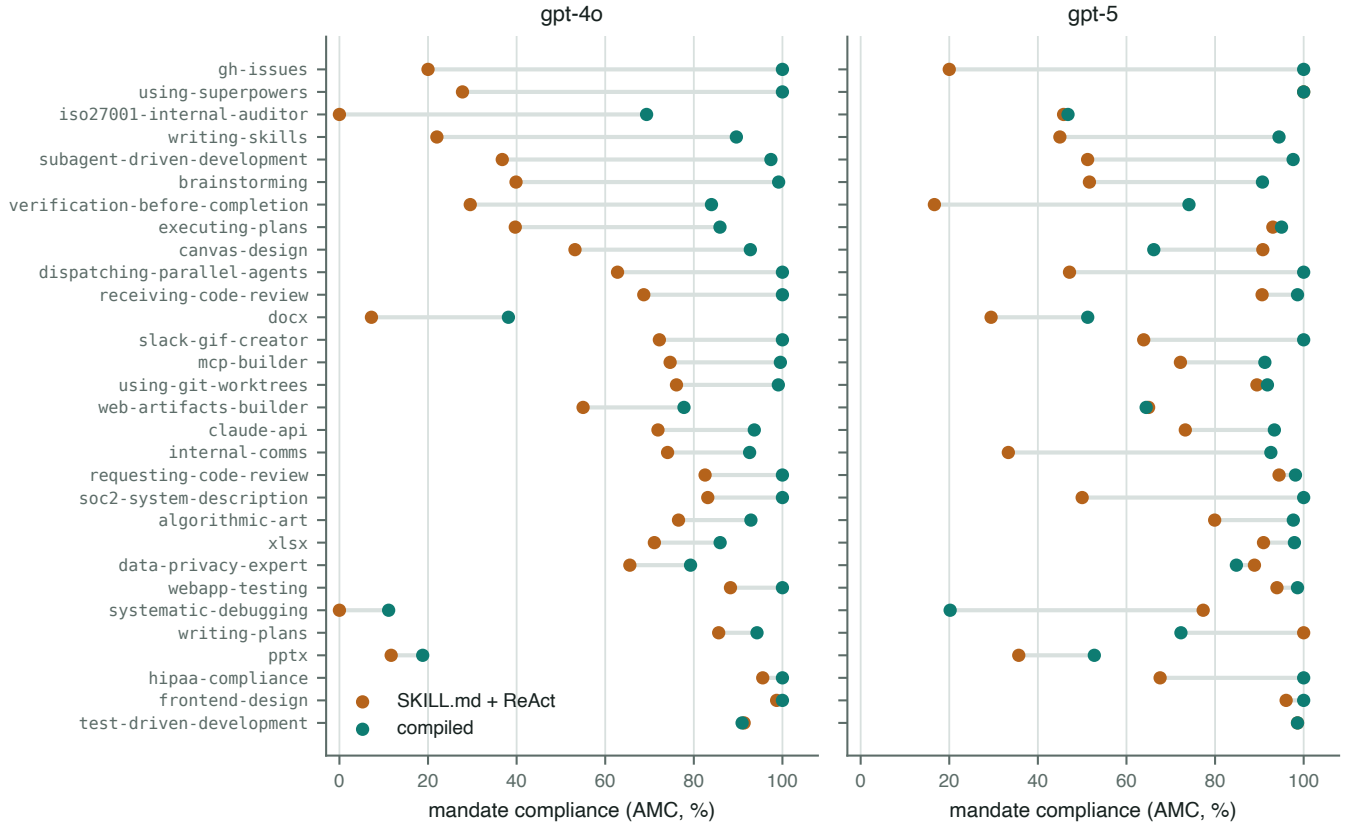


Figure 5: Per-skill procedure execution on both models. Each line joins the prose result to the harness result for one skill, sorted by gap. The advantage is largest on mechanism-heavy skills and smallest on judgment-heavy ones.

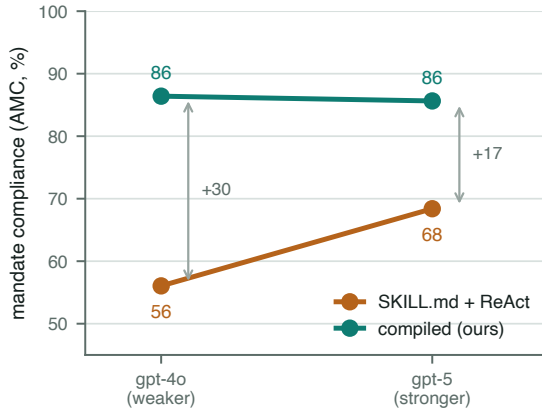


Figure 6: Procedure execution against model capability. The compiled harness is flat across the two models because the graph, not the model, carries the procedure. The prose agent improves with capability, so the advantage of compiling is largest on the weaker model.

compliance metric records. Cost and procedure execution are therefore not independent: some of the prose arm’s apparent economy is the cost of work it did not do.

Key Takeaways.

- (1) **Compilation is not a tax.** The harness is cheaper on 24 of 32 skills and 0.58× at the median, so the reliability gain does not have to be paid for.
- (2) **Some savings are illusory.** Where the harness costs more, it is running an adaptive loop the prose agent skipped, so part of prose’s economy is unperformed work.

5.6 RQ4: what the compiler enforces

Finally, STRUCT-COV reports, per skill, how each mandatory step is realized in the emitted module. Figure 8 shows the distribution. Mechanism-heavy skills lower predominantly to code-owned gates, which are the mandates that hold regardless of model. Judgment-heavy skills retain a large model-owned residue, which is the Owner test operating as designed: a step that requires taste cannot be converted into a guarantee, and pretending otherwise would produce a harness that constrains the model where it should not.

Importantly, the diagnostic value is in the remaining two categories. A monolithic realization is the failure mode that most resembles success, because the compilation completes and the mandate is nominally present, yet the step retains exactly the risk profile it had as prose. During development, these reports were the mechanism by which we found and repaired compilations that had silently



Figure 7: Tokens per run, harness relative to prose, on a logarithmic scale. The harness is cheaper on 24 of 32 skills, with a median of 0.58x. The costly minority are skills built around an adaptive tool-using loop that the harness faithfully runs and the prose agent often skipped.

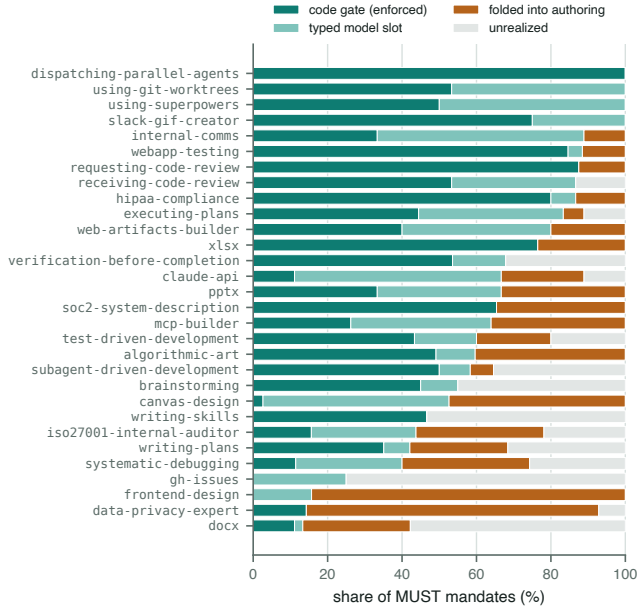


Figure 8: STRUCT-COV by skill: the share of mandatory steps realized as a code-owned gate, a dedicated model slot, folded into another slot, or unrealized. Mechanism-heavy skills lower predominantly to gates; judgment-heavy skills retain a large model-owned residue.

folded a mandated verification step into a neighboring authoring

slot. Because STRUCT-COV names the rule and the node, the repair is local.

Key Takeaways.

- (1) **Enforceable share is a property of the skill.** How much of a skill compiles to guarantees is determined by how much of its procedure is determined, which varies widely across families.
- (2) **The dangerous outcome is the one that looks fine.** A mandate folded into another slot compiles cleanly while remaining model-dependent, which is why it is a gate failure rather than a warning.

5.7 Threats to validity

Gate credit. The most direct objection to our measurement is that the harness receives credit for code-owned gates on the evidence of its own node trace, which prose has no equivalent of. To test whether the result is an artifact of this decision, we recompute the comparison using only the mandates the judge decides from the trace, with no automatic credit for gates. The harness still leads substantially, 77% against 56%. The advantage is therefore not a scoring convention; it is the same steps being performed more often.

Judged measurement. However, mandate statuses are assigned by a model-based judge, which introduces noise. We mitigate this by constraining the judge to a four-way typed decision per mandate against a fixed reference procedure, rather than allowing free-form assessment, and by excluding unreached mandates from the denominator so that branching does not have to be adjudicated.

Skill sample. Additionally, our 30 skills are weighted toward document production, software process, and compliance work. Skills in other domains may have different mechanism-to-judgment ratios, and the Owner test predicts that the benefit of compiling would move accordingly.

Two models. The capability axis is measured over two models from one provider. The claim it supports is that the harness result is invariant while the prose result is not, which two points establish; the shape of the prose curve across a wider range of capability remains open.

6 Related Work

Broadly, systems that address unreliable agent behavior fall into two categories: those that retain the prose specification and add machinery around it, and those that change the substrate in which the procedure is expressed. Our work belongs to the latter category.

Watching the agent. A substantial line of work supervises a black-box agent at run time. Runtime verification attaches temporal-logic monitors to agent executions [1], while GuardAgent [9] and ShieldAgent [3] evaluate or guard actions at the tool boundary, and AgentSpec [8] provides a domain-specific language of symbolic rules checked against each run. While these systems offer increasingly sophisticated oversight and are effective at preventing prohibited actions, they differ from our work in what they can accomplish. Specifically, a monitor can veto a wrong action but

cannot cause a mandated action to occur, and procedural work requires that the prescribed steps happened, not merely that bad ones were filtered. Their rules are also authored separately from the skill they police, so the specification and its enforcement drift as either changes. Our approach differs in that we compile the specification into the executing program rather than checking a separate rule set against it.

Orchestration frameworks. In contrast, frameworks such as LangChain and related agent toolkits provide composition, memory, and tool plumbing, and they materially reduce the effort of building an agent. The procedure itself, however, still resides in prose that the model interprets at run time, so the properties we target are unaffected: a framework that sequences calls does not prevent a model from skipping a mandated check inside one of them. These systems are complementary to ours and can host a compiled harness as readily as a prose agent.

Structured generation and typed model calls. Separately, a parallel line of work constrains what a model emits. Structured-output and constrained-decoding techniques enforce a schema on a response, and meaning-typed programming [4] raises this to a language abstraction in which a function’s body is delegated to a model with types carrying the intent, which is the mechanism we use for every model-owned slot. Prompt-programming systems such as DSPy and LMQL optimize and structure individual calls. All of these type or optimize a call; none of them encode a multi-step procedure, which is the unit a skill specifies. We build on typed model calls as the state of the art for the leaves of our graph, and contribute the representation and compilation of the procedure that connects them.

Agentic execution substrates. Object-Spatial Programming [6] provides the graph and walker abstractions that our Lowering stage targets, in which computation moves to data along typed topologies. Our contribution is not the substrate but the Extraction stage: a method for obtaining a faithful program in it from a document that a domain expert wrote in prose.

Program synthesis from natural language. Compiling prose to programs is a longstanding goal, and modern code generation performs it routinely for self-contained functions. Our setting differs in what faithfulness requires. A synthesized function is judged by whether its behavior satisfies a specification, whereas a compiled skill is judged by whether the resulting program preserves the source document’s obligations, including their binding force, and by whether the steps the source mandates demonstrably execute. This is why Extraction is organized around grounding, coverage, and deontic auditing rather than around test satisfaction.

7 Conclusion

Agents increasingly acquire specialized capability from prose skills, but a prose skill is only described to the runtime and never encoded in it, so the steps it mandates are re-derived, and sometimes silently dropped, on every run. This paper introduced **Skill Compilation**, an approach that treats a prose skill as source code and compiles it into an executable harness, and realized it in **SIGIL**, a compiler built around **AG-IR**, a typed agentic intermediate representation

that separates model-owned cognition from code-owned mechanism and carries provenance from each compiled ability back to the sentence that mandated it. A judgment Extraction stage, held to the source by a grounded specification loop and six compile gates, produces the IR; a deterministic Lowering stage translates it with no further choices. Across 30 skills and two models, compiled harnesses perform 86% of mandated steps against 56% for prose, complete the whole procedure 2.3 times as often, and cost 0.58× the tokens at the median, while holding at 86% on both a weaker and a stronger model where prose swings from 56% to 68%. Structure substitutes for capability exactly where models are weakest and cheapest. Future work includes automated repair of mandates that lower monolithically, and compiling skills specifically for small on-device models, where the capability axis suggests the benefit is largest.

References

- [1] Parand A. Alamdari, Toryn Q. Klassen, and Sheila A. McIlraith. 2026. Formal Methods Meet LLMs: Auditing, Monitoring, and Intervention for Compliance of Advanced AI Systems. arXiv:2605.16198 [cs.AI]
- [2] Anthropic. 2025. Equipping Agents for the Real World with Agent Skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>. Accessed July 2026.
- [3] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.
- [4] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothyathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2, Article 314 (2025). doi:10.1145/3763092
- [5] Jaseci Labs. 2024. The Jac Programming Language and Jaseci Stack. <https://www.jac-lang.org>.
- [6] Jason Mars. 2025. Object-Spatial Programming. arXiv:2503.15812 [cs.PL]
- [7] OpenAI. 2025. GPT-5 System Card. <https://openai.com/index/gpt-5-system-card/>.
- [8] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*.
- [9] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.
- [10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.